

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій

ЗВІТ
з лабораторної роботи №1
з дисципліни “Професійний Python”
на тему “Організація професійного Python-проєкту та Git”

Виконала
студентка 3 курсу
групи Феп-33с
Корхова Марія Сергіївна

Перевірів
професор
Свелеба Сергій Андрійович

Львів 2026

Тема роботи: Організація професійного Python-проєкту та Git.

Мета роботи: Ознайомитися з основними принципами організації професійного Python-проєкту, навчитися створювати ізольоване середовище розробки, формувати структуру проєкту за принципом src-layout, працювати з модулями та пакетами Python, створювати файл `pyproject.toml`, використовувати Git для контролю версій та оформлювати базову документацію проєкту.

Номер індивідуального варіанта: 7.

Постановка задачі

Створити консольний Python-застосунок "Система оцінювання студентів" (Варіант 7) з використанням архітектури src-layout. Програма має зберігати такі дані про студента: ПІБ, дисципліну та оцінки. Необхідно реалізувати бізнес-логіку для: обчислення середнього бала, знаходження мінімальної та максимальної оцінок, визначення успішності, сортування студентів, пошуку студентів із середнім балом вище заданого.

Технічні вимоги до реалізації включають: розподіл програми щонайменше на три модулі (data model, business logic, точка входу `main.py`), використання анотацій типів (type hints) та конструкції `if __name__ == "__main__":`. Проєкт має бути налаштований через `pyproject.toml`, містити базову документацію у `README.md`, ігнорувати тимчасові файли через `.gitignore` та контролюватися за допомогою системи Git.

Структура проєкту

Проект побудовано за принципом `src-layout`, що дозволяє відокремити конфігураційні файли від програмного коду, який розміщено у пакеті `lab01` всередині каталогу `src/`.

Lab01/

```
|—.venv/
|—.gitignore
|— pyproject.toml
|— README.md
|— report/
|   └─ MariiaKorkhovaLab01FEP33.pdf
└─ src/
    └─ lab01/
        ├── init.py
        ├── main.py
        ├── models.py
        └─ services.py
```

Опис створених **modules**

`models.py`: Містить `data model Student`. Використовує декоратор `@dataclass` для автоматичної генерації конструктора та зберігання стану об'єкта (ПІБ, дисципліна, список оцінок).

services.py: Ізолює business logic від вводу/виводу. Містить набір функцій для обробки даних (обчислення середнього бала, визначення мін/макс оцінки, фільтрація, сортування).

main.py: Точка входу програми (application entry point). Ініціалізує демо-дані, викликає функції з services.py для обробки та виводить результат. Містить обов'язкову конструкцію `if __name__ == "__main__":`.

__init__.py: Маркерний файл, який перетворює каталог lab01 на Python package.

8. Текст основних фрагментів коду

Повний код файлу src/lab01/main.py

```
from lab01.models import Student
from lab01.services import (
    get_min_max_grades,
    is_successful,
    sort_students,
    filter_by_min_average
)

def main() -> None:
    students = [
        Student("Олег Мельник", "Бази даних", [75.0, 80.0, 60.0]),
        Student("Анна Коваль", "Комп'ютерні мережі", [50.0, 45.0, 55.0]),
        Student("Марія", "Професійний Python", [95.0, 98.0, 100.0])
    ]

    print("Рейтинг студентів:")
    for s in sort_students(students):
        min_g, max_g = get_min_max_grades(s)
```

```

        status = "Зараховано" if is_successful(s) else "Не зараховано"
        print(f"{s.full_name} | Сер: {s.average_grade:.1f} | Мін:
{min_g}, Макс: {max_g} | {status}")

    min_threshold = 85.0
    print(f"\nСтуденти з балом вище {min_threshold}:")
    for s in filter_by_min_average(students, min_threshold):
        print(f"- {s.full_name} ({s.subject})")

if __name__ == "__main__":
    main()

```

Повний код файлу src/lab01/services.py

```

from lab01.models import Student

def get_min_max_grades(student: Student) -> tuple[float, float]:
    if not student.grades:
        return 0.0, 0.0
    return min(student.grades), max(student.grades)

def is_successful(student: Student, passing_score: float = 60.0) ->
bool:
    return student.average_grade >= passing_score

def sort_students(students: list[Student]) -> list[Student]:
    return sorted(students, key=lambda s: s.average_grade, reverse=True)

def filter_by_min_average(students: list[Student], min_average: float)
-> list[Student]:
    return [s for s in students if s.average_grade > min_average]

```

Повний код файлу src/lab01/models.py

```

from dataclasses import dataclass

```

```

@dataclass
class Student:
    full_name: str
    subject: str
    grades: list[float]

    @property
    def average_grade(self) -> float:
        return sum(self.grades) / len(self.grades) if self.grades else
0.0

```

Пояснення pyproject.toml

Файл містить конфігурацію та метадані Python-проєкту, необхідні для його встановлення як пакета. Опис секцій:

[build-system]: Визначає setuptools як build-backend (систему збирання), яка використовується для пакування проєкту.

[project]: Містить основні метадані: назву пакета (name = "lab01"), версію та строгу вимогу до версії інтерпретатора (requires-python = ">=3.11").

[project.scripts]: Реєструє консольну команду lab01, яка при виклику в терміналі автоматично запускає функцію main з модуля lab01.main. Це уможливорює виконання програми як повноцінного застосунку після встановлення в editable mode (pip install -e .).

Вміст .gitignore

```

.venv/
__pycache__/

```

```
*.pyc
dist/
build/
*.egg-info/
```

Фрагмент README.md

```
# Lab01 - Student Evaluation System
Console application for tracking student grades and calculating academic
statistics.

## Requirements
Python 3.11+

## Installation & Run
```bash
python3 -m venv .venv
source .venv/bin/activate
python3 -m pip install -e .
lab01
```

## Author

Марія Корхова
Group: ФЕП-33
```

Скриншот запуску application

```
(.venv) korkhovamariia@MariiaKorkhova:~/Lab01$ lab01
Рейтинг студентів:
Марія | Сер: 97.7 | Мін: 95.0, Макс: 100.0 | Зараховано
Олег Мельник | Сер: 71.7 | Мін: 60.0, Макс: 80.0 | Зараховано
Анна Коваль | Сер: 50.0 | Мін: 45.0, Макс: 55.0 | Не зараховано

Студенти з балом вище 85.0:
– Марія (Професійний Python)
```

Результат git status

```
(.venv) korkhovamariia@MariiaKorkhova:~/Lab01$ git status
On branch master
nothing to commit, working tree clean
```

Результат git log --oneline

```
(.venv) korkhovamariia@MariiaKorkhova:~/Lab01$ git log --oneline
e5506fc (HEAD -> master) Add application entry point and documentation
75976d4 Add student model and business logic
8910fd6 Create project structure
(.venv) korkhovamariia@MariiaKorkhova:~/Lab01$ |
```

Контрольні запитання

1. Python-проект — це організована структура файлів та каталогів, яка містить програмний код, тести, зовнішні залежності та конфігураційні файли (наприклад, pyproject.toml, README.md, .gitignore).
2. Відмінність проекту від скрипта полягає у тому, що навчальна програма (скрипт) часто складається з одного файла, тоді як для професійної розробки програму поділяють на логічні компоненти через збільшення кількості функцій, класів та тестів.
3. Python module — це звичайний файл із розширенням .py, який містить код.
4. Python package — це каталог із Python-модулями.

5. Файл `__init__.py` потрібний для того, щоб вказати інтерпретатору Python, що даний каталог є пакетом (package), і дозволити імпорт модулів із нього.
6. `src-layout` — це структура проєкту, у якій програмний код (пакети та модулі) розміщується в окремому підкаталозі `src/`.
7. Переваги `src-layout` полягають у чіткому відокремленні конфігураційних файлів, віртуального середовища та тестів від безпосереднього коду застосунку.
8. `Virtual environment` використовується для ізоляції бібліотек та залежностей одного Python-проєкту від інших проєктів на комп'ютері.
9. Створити `.venv` можна за допомогою команди `python -m venv .venv`.
10. Активувати `virtual environment` можна командою `.venv\Scripts\activate` у Windows або `source .venv/bin/activate` у Linux/macOS.
11. `pip` використовується для встановлення зовнішніх пакетів (packages) та встановлення локального проєкту.
12. Команда `python -m pip` рекомендується для того, щоб гарантувати виклик менеджера пакетів `pip` саме для того середовища Python, яке є зараз активним.
13. Файл `pyproject.toml` містить метадані та конфігурацію Python-проєкту, зокрема вимоги до системи збирання та точки входу.
14. Секція `[project]` містить назву проєкту (name), версію (version), опис (description) та вимоги до мінімальної версії Python (requires-python).
15. `Editable installation` (команда з параметром `-e`) — це режим встановлення пакета, який дозволяє вносити зміни до програмного коду без необхідності його перевстановлення.

16. Конструкція `if __name__ == "__main__":` означає, що головна функція буде виконана тільки тоді, коли файл запускається безпосередньо, а не під час його імпорту як модуля.
17. Git — це розподілена система контролю версій (distributed version control system), яка дозволяє зберігати історію змін, працювати в команді та створювати гілки.
18. Repository — це сховище, де зберігається історія змін, файли проєкту та відстежується авторство.
19. Commit — це збережений логічний етап розробки в історії репозиторію.
20. Команда `git status` використовується для перегляду стану репозиторію та відстеження незбережених чи змінених файлів.
21. Команда `git add` потрібна для додавання файлів до списку підготовлених перед створенням коміту.
22. Команда `git commit` потрібна для фіксації підготовлених змін у репозиторії разом із описовим повідомленням (-m).
23. Команда `git log` показує історію створених комітів.
24. Файл `.gitignore` визначає файли (наприклад, кеші, тимчасові файли, налаштування IDE, секрети), які не потрібно додавати до Git-репозиторію.
25. `.venv` не потрібно комітити, оскільки віртуальне середовище містить локальні бінарні файли та залежності, які можна легко відтворити на іншій машині, і вони лише засмічують історію та об'єм репозиторію.
26. README описує проєкт, його вимоги, процес інсталяції, команди для запуску застосунку та дані автора.
27. Type hints — це підказки типів у коді (наприклад, `-> None` або `list[Student]`), які покращують зрозумілість коду та дозволяють аналізаторам перевіряти типи.

- 28.Програму розділяють на modules для того, щоб відокремити введення/виведення, обчислення, моделі даних, бізнес-логіку, конфігурацію та тести.
- 29.Business logic — це окремий модуль, який реалізує основні правила обробки даних (наприклад, обчислення середнього бала або пошук найуспішнішого студента).
- 30.Відокремлювати application entry point від business logic рекомендовано для того, щоб модулі з логікою можна було вільно імпортувати в інші частини програми або тестові файли без автоматичного запуску всієї програми.

Висновки

У результаті виконання лабораторної роботи здійснено перехід від написання ізольованих Python-скриптів до організації структурованого програмного проєкту. Розроблено консольний застосунок для оцінювання студентів на базі архітектури src-layout із чітким розділенням коду на моделі даних, бізнес-логіку та точку входу програми.

Проєкт налаштовано для роботи в ізольованому віртуальному середовищі з використанням конфігураційного файлу pyproject.toml для керування метаданими та встановлення пакета в editable mode. Забезпечено версіонування коду за допомогою системи Git із фіксацією логічних етапів розробки та створено базову проєктну документацію (README.md). Застосовані інструменти, практики ізоляції залежностей та модульна архітектура формують необхідний фундамент для подальшого інтегрування

складних структур даних, об'єктно-орієнтованого програмування, автоматизованого тестування та CI/CD у наступних лабораторних роботах.